

Propositional Logic

Mingxun Zhou (mingxunz@ust.hk)

COMP2711 Discrete Mathematical Tools

1 Motivation

Imagine that your first task on a security team is to program a laboratory door. The requirement is simple:

The door opens only when the card is valid and the entered PIN is correct.

A rushed implementation uses the following decision:

```
if card OR pin: open
```

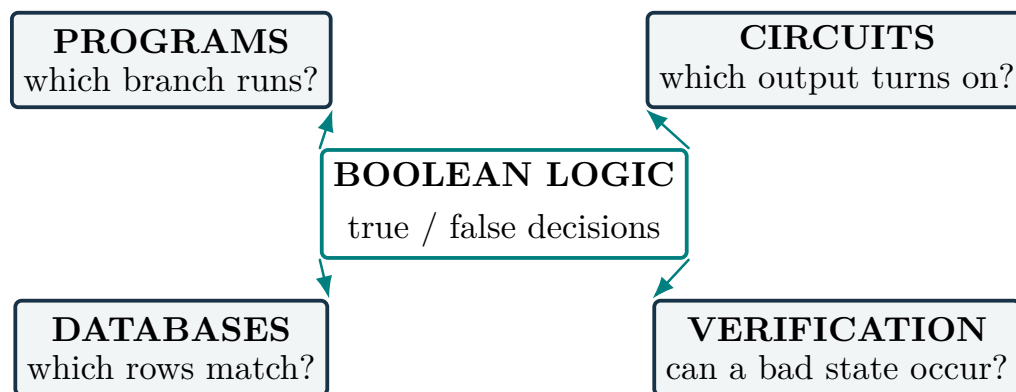
The program runs, but the rule is unsafe: a valid card alone is enough, and a correct PIN alone is enough. Replacing one word gives the intended decision:

```
if card AND pin: open
```

This is a small bug with a large consequence. The computer follows the logical structure exactly; it does not recover the programmer's intention.

1.1 Logic in computer science

Propositional logic is the language of true-or-false decisions. The same structure appears throughout computer science:



A requirement can be translated into a formula. The formula can be evaluated by a program or a circuit. A truth table can then check every possible input and expose a state that violates the requirement.

Logic also remains fundamental to mathematics. It separates assumptions from conclusions, distinguishes a statement from its converse, recognizes equivalent formulations, and finds counterexamples.

1.2 Boole and Shannon

In 1847, George Boole published *The Mathematical Analysis of Logic*. His central move was to treat logical reasoning as symbolic calculation. In 1937, while completing his MIT master's thesis, Claude Shannon recognized that relay switches obeyed the same algebra: a statement could be true or false, and a switch could be on or off. Complicated circuits could therefore be analyzed and simplified symbolically before construction.¹

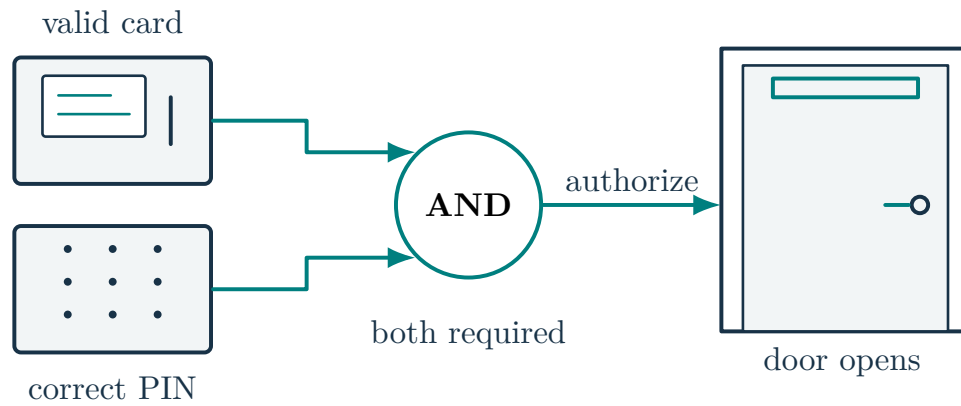
Boole turned logic into mathematics; Shannon used that mathematics to reason about machines. Modern programs, digital circuits, database queries, and verification tools still use the same true-or-false structure.

1.3 The lab door

The laboratory door will be our first running example. Its intended rule is:

The door opens exactly when the card is valid and the entered PIN is correct.

¹See the [Library of Congress](#) record for Boole's 1847 work and the [MIT](#) account of Shannon's thesis.



The sentence is easy to understand informally. Logic lets us answer precise questions:

- Does it prevent the door from opening with only one credential?
- Does it require the door to open when both credentials are correct?
- How could a computer check every possible state?

We will not write the final formula yet. We first need to build the language that makes the rule precise.

2 Formulas and truth tables

2.1 Propositions

MUST UNDERSTAND

Proposition. A declarative sentence with exactly one truth value: true or false.

Sentence	Proposition?	Reason
HKUST is in Hong Kong.	yes	It is true.
17 is divisible by 3.	yes	It is false.
$x + 2 = 7$.	not yet	The value of x is unspecified.
Submit the assignment.	no	It is a command.
Is the server online?	no	It is a question.

Truth and propositionhood are different questions. A false sentence can still be a proposition if its truth value is definite.

MUST UNDERSTAND

Atomic proposition. One complete claim treated as a single unit.

For the door example, define

$card$ = “the card is valid”, pin = “the PIN is correct”, $open$ = “the door opens”.

Each variable names a complete proposition, not an object or number.

2.2 Formulas

MUST UNDERSTAND

Logical formula. An atomic proposition, or propositions combined by logical connectives.

Arithmetic formula	Logical formula
$(x + y) - z$	$(p \vee q) \wedge \neg r$
x, y, z are numbers	p, q, r are T or F
rules: $+, -, \times, \div$	connectives: $\neg, \wedge, \vee, \oplus$

The pattern is the same: variables are combined by rules. Here the rules are NOT (negation), AND, OR, and XOR. A proposition variable by itself is also a logical formula.

The construction is recursive:

1. Every atomic proposition is a formula.
2. If A is a formula, then $\neg A$ is a formula.

3. If A and B are formulas, then $(A \wedge B)$ and $(A \vee B)$ are formulas.

MUST UNDERSTAND

Connective. An operation that combines one or more formulas to form a new formula.

We first build formulas with $\neg$, $\wedge$, and $\vee$. We define $\oplus$ from these symbols.

Name	Symbol	Read as	True when
negation	$\neg A$	not A	A is false
conjunction	$A \wedge B$	A and B	both are true
disjunction	$A \vee B$	A or B	at least one is true
exclusive disjunction	$A \oplus B$	A XOR B	exactly one is true

The ordinary disjunction $A \vee B$ is **inclusive**: it remains true when both inputs are true. Exclusive disjunction, also called **exclusive OR** or **XOR**, excludes that case. It is the derived connective

$$A \oplus B := (A \vee B) \wedge \neg(A \wedge B).$$

The credential condition for the door is now a formula:

$$card \wedge pin.$$

It is true exactly when both credentials are correct. We still do not have symbols for the complete rule “the door opens exactly when ...”; those will come later.

Formula structure

Consider

$$(p \vee q) \wedge \neg p.$$

Its main connective is $\wedge$. The two immediate subformulas are $p \vee q$ and $\neg p$. This tree-like structure tells us the order of evaluation: compute the inner pieces, then the main connective.

Parentheses carry meaning. The formulas

$$(p \vee q) \wedge r \quad \text{and} \quad p \vee (q \wedge r)$$

are different: the first requires r ; the second does not.

2.3 Truth tables

A **truth assignment** chooses a truth value for every atomic proposition. With n atoms there are 2^n assignments, because every atom has two choices.

The truth tables make the inclusive–exclusive distinction explicit:

p	$\neg p$	p	q	$p \wedge q$	$p \vee q$	$p \oplus q$
T	F	T	T	T	T	F
T	F	T	F	F	T	T
F	T	F	T	F	T	T
F	T	F	F	F	F	F

Thus $p \vee q$ means “at least one,” whereas $p \oplus q$ means “exactly one.”

To evaluate $(p \vee q) \wedge \neg p$, compute its subformulas from the inside out:

p	q	$p \vee q$	$\neg p$	$(p \vee q) \wedge \neg p$
T	T	T	F	F
T	F	T	F	F
F	T	T	T	T
F	F	F	T	F

The final column is true in exactly one case: p is false and q is true.

Truth-table construction

1. List the atoms and create 2^n input rows.
2. Add a column for each useful subformula.
3. Evaluate inner connectives before outer connectives.
4. Compute the main connective last.
5. Read the answer from the final column.

A truth table is slow but decisive: it checks every possible case.

2.4 Formula classification

MUST UNDERSTAND

Tautology and contradiction. A tautology is true on every assignment; a contradiction is false on every assignment.

Tautologies and contradictions behave like constant arithmetic expressions: their outputs do not depend on their inputs. A contingency, by contrast, changes value as its inputs change.

Behavior	Arithmetic example	Logical examples
always 1	$x - x + 1$	tautologies: $p \vee \neg p$, $q \vee \neg q$
always 0	$x - x$	contradictions: $p \wedge \neg p$, $q \wedge \neg q$
changes	$x + 1$	contingencies: $p \wedge q$, $p \vee q$

To show that a formula is not a tautology, one false row is enough. That row is a counterexample. To prove a tautology by truth table, every row must be checked.

2.5 Logical equivalence

MUST UNDERSTAND

Logical equivalence. Two formulas are logically equivalent, written $A \equiv B$, when they have the same truth value on every assignment.

This is analogous to an arithmetic identity:

$$(x + 1)(x - 1) = x^2 - 1.$$

We do not choose one particular value of x . The two arithmetic expressions have the same value for every x . Similarly, logically equivalent formulas have the same truth value for every truth assignment; we do not choose one particular truth-table row.

For example:

p	q	$\neg(p \wedge q)$	$\neg p \vee \neg q$
T	T	F	F
T	F	T	T
F	T	T	T
F	F	T	T

The columns match, so

$$\neg(p \wedge q) \equiv \neg p \vee \neg q.$$

This is De Morgan's law: "not both" means "at least one is not."

The symbol $\equiv$ does not build a larger formula. It records a judgment about two formulas: they agree in every possible case. We can prove this judgment by comparing truth-table columns or, later, by transforming one formula with known equivalence laws. The laws are proof tools; they are not the definition of equivalence.

3 Conditionals

3.1 Implication

MUST UNDERSTAND

Implication. The formula $p \rightarrow q$ means that whenever p is true, q must be true. It is read "if p , then q ."

Consider the rule

if I miss the bus, then I am late for class.

Let p mean "I miss the bus" and let q mean "I am late for class." The four possible cases are:

p	q	$p \rightarrow q$	What happened?
T	T	T	Missed the bus and was late; the rule holds.
T	F	F	Missed the bus but was not late; the rule fails.
F	T	T	Did not miss the bus; the rule was not tested.
F	F	T	Did not miss the bus; the rule was not tested.

There is exactly one false case: p is true and q is false. When p is false, the implication makes no requirement about q , so propositional logic assigns the implication the value T.

This convention can make a strange sentence formally true:

“If the world ended yesterday, then I am an alien.”

The if-part is false, so the implication is true whether or not the conclusion is true. This is a statement about its propositional truth value, not evidence of a causal connection or a believable conclusion.

Now that equivalence is defined, we can compare implication with an older formula:

$$\boxed{p \rightarrow q \equiv \neg p \vee q.}$$

Both are false exactly when $p = \text{T}$ and $q = \text{F}$.

The first two door requirements can now be written:

$$\text{open} \rightarrow \text{card}, \quad \text{open} \rightarrow \text{pin}.$$

An open door must have both a valid card and a correct PIN.

Necessary and sufficient conditions

In $p \rightarrow q$, p is **sufficient** for q , and q is **necessary** for p .

For example,

$$4 \mid n \rightarrow 2 \mid n.$$

Divisibility by 4 is sufficient for evenness. Evenness is necessary for divisibility by 4. Evenness is not sufficient: 6 is a counterexample.

Converse and contrapositive

Name	General form	Example for p : “It is a dog,” q : “It is an animal”
original	$p \rightarrow q$	A dog is an animal, true
converse	$q \rightarrow p$	An animal is a dog, false: it could be a cat
inverse	$\neg p \rightarrow \neg q$	Not a dog means not an animal, false: it could be a cat
contrapositive	$\neg q \rightarrow \neg p$	Not an animal means not a dog, true

A cat disproves both the converse and the inverse. The contrapositive is equivalent to the original:

$$\boxed{p \rightarrow q \equiv \neg q \rightarrow \neg p.}$$

3.2 Biconditional

MUST UNDERSTAND

Biconditional. The formula $A \leftrightarrow B$ is true exactly when A and B have the same truth value. It is read “ A if and only if B .”

Like every formula, a biconditional receives a truth value only after an assignment v is chosen:

$$v(A \leftrightarrow B) = \text{T} \quad \text{exactly when} \quad v(A) = v(B).$$

Thus the biconditional asks whether A and B agree in the current case. Its answer may be true or false.

p	q	$p \leftrightarrow q$
T	T	T
T	F	F
F	T	F
F	F	T

A biconditional is equivalent to two implications:

$$A \leftrightarrow B \equiv (A \rightarrow B) \wedge (B \rightarrow A).$$

One direction proves sufficiency; the other proves necessity.

It can also be written without any arrows:

$$A \leftrightarrow B \equiv (A \wedge B) \vee (\neg A \wedge \neg B).$$

The arrow-free form says exactly what the truth table shows: either both formulas are true, or both formulas are false.

We can finally express the original door rule:

$$open \leftrightarrow (card \wedge pin).$$

The left-to-right direction prevents unauthorized opening. The right-to-left direction requires the door to open when both credentials are correct.

Biconditional and logical equivalence

The two symbols have different jobs:

Notation	What it is	What it says
$A \leftrightarrow B$	a compound formula	In one particular case: do A and B agree? Once values are assigned, the formula is T or F.
$A \equiv B$	a judgment about formulas	For every assignment, A and B agree.

Their exact relationship is

$$A \equiv B \quad \text{exactly when} \quad A \leftrightarrow B \text{ is always T.}$$

Here “always” means every possible assignment to whatever atomic propositions occur in A or B . We do not need to name particular variables or choose a particular assignment. If A and B agree on every assignment, their biconditional is always true. The converse is the same reasoning in reverse. Thus $\leftrightarrow$ tests agreement in one case; $\equiv$ asserts agreement in every case.

3.3 Translation

A disciplined translation method prevents most direction errors:

1. Define short, positive atomic propositions.
2. Mark the structural words: and, or, not, if, only if, iff.
3. For each implication, identify the trigger and the required consequence.
4. Add parentheses, then read the formula back in English.
5. Test one state that should violate the rule.

Useful patterns are:

English	Formula
p if q	$q \rightarrow p$
p only if q	$p \rightarrow q$
p iff q	$p \leftrightarrow q$
neither p nor q	$\neg p \wedge \neg q$
not both p and q	$\neg(p \wedge q)$

Lab door rule

The verbal rule contains three requirements:

$$(open \rightarrow card) \wedge (open \rightarrow pin) \wedge ((card \wedge pin) \rightarrow open).$$

Combine the two consequences of $open$:

$$(open \rightarrow card) \wedge (open \rightarrow pin) \equiv open \rightarrow (card \wedge pin).$$

The complete rule becomes

$$(open \rightarrow (card \wedge pin)) \wedge ((card \wedge pin) \rightarrow open) \\ \equiv \boxed{open \leftrightarrow (card \wedge pin)}.$$

Its truth table identifies every correct state and every system bug:

$card$	pin	$open$	$open \leftrightarrow (card \wedge pin)$	Interpretation
T	T	T	T	correct opening
T	T	F	F	functionality failure
T	F	T	F	opens with wrong PIN
T	F	F	T	correctly closed
F	T	T	F	opens with invalid card
F	T	F	T	correctly closed
F	F	T	F	opens without credentials
F	F	F	T	correctly closed

4 Problem solving

4.1 Worked equivalence problem

Show that

$$(p \rightarrow q) \wedge (p \rightarrow \neg q) \equiv \neg p.$$

The task is to prove that the formula on the left and $\neg p$ have the same truth value for every assignment to p and q .

Natural-language reasoning

To read the formula concretely, temporarily let p mean “the door opens” and let q mean “the alarm sounds.” The left side then contains two rules:

If the door opens, then the alarm sounds.

If the door opens, then the alarm does not sound.

Suppose the door opens. The first rule requires the alarm to sound, while the second requires it not to sound. These requirements cannot both be met. Therefore, both rules can be true only when the door stays closed, which means p is false.

When the door stays closed, both implications are true regardless of the alarm. Hence the left side is true exactly when $\neg p$ is true. This concrete reading explains the expected answer; we next give two formal proofs for arbitrary propositions p and q .

Two solution methods

Method	What to do
truth table	Check all four assignments and compare the two final columns.
equivalence laws	Rewrite the left side step by step until it becomes $\neg p$.

Truth-table solution

p	q	$\neg q$	$p \rightarrow q$	$p \rightarrow \neg q$	$(p \rightarrow q) \wedge (p \rightarrow \neg q)$	$\neg p$
T	T	F	T	F	F	F
T	F	T	F	T	F	F
F	T	F	T	T	T	T
F	F	T	T	T	T	T

The last two columns match on every row. Therefore,

$$(p \rightarrow q) \wedge (p \rightarrow \neg q) \equiv \neg p.$$

Equivalence-law solution

An algebraic calculation replaces an arithmetic expression with an equal expression. A calculation in **logical arithmetic** replaces a formula with a logically equivalent formula. Every step preserves the truth value on every assignment.

The solution needs four laws:

Purpose	Law
remove arrows	$A \rightarrow B \equiv \neg A \vee B$
	$p \rightarrow \neg q \equiv \neg p \vee \neg q$
distribute	$(A \vee B) \wedge (A \vee C) \equiv A \vee (B \wedge C)$
expose a contradiction	$q \wedge \neg q \equiv \text{F}$
remove F	$A \vee \text{F} \equiv A$

Distributive law

The distributive law can be used in either direction:

$$A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C).$$

For this problem, read it from right to left. Match

$$A = \neg p, \quad B = q, \quad C = \neg q.$$

Then

$$(\neg p \vee q) \wedge (\neg p \vee \neg q) \equiv \neg p \vee (q \wedge \neg q).$$

The repeated part $\neg p$ plays the role of A in both parentheses.

Now calculate:

$$\begin{aligned}
 (p \rightarrow q) \wedge (p \rightarrow \neg q) &\equiv (\neg p \vee q) \wedge (\neg p \vee \neg q) && \text{remove arrows} \\
 &\equiv \neg p \vee (q \wedge \neg q) && \text{distribute} \\
 &\equiv \neg p \vee \text{F} && \text{contradiction} \\
 &\equiv \boxed{\neg p} && \text{remove F.}
 \end{aligned}$$

Each line changes one part, preserves the truth value, and names the law used.

Equivalence-law reference

The following tables collect the standard laws used in this lecture. Each law may be applied from left to right or from right to left.

Law	Equivalent formulas
identity	$p \wedge \mathbf{T} \equiv p; p \vee \mathbf{F} \equiv p$
domination	$p \vee \mathbf{T} \equiv \mathbf{T}; p \wedge \mathbf{F} \equiv \mathbf{F}$
idempotent	$p \vee p \equiv p; p \wedge p \equiv p$
double negation	$\neg\neg p \equiv p$
complement	$p \vee \neg p \equiv \mathbf{T}; p \wedge \neg p \equiv \mathbf{F}$
commutative	$p \vee q \equiv q \vee p; p \wedge q \equiv q \wedge p$
associative	$(p \vee q) \vee r \equiv p \vee (q \vee r)$ $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$
distributive	$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$ $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$
De Morgan	$\neg(p \wedge q) \equiv \neg p \vee \neg q$ $\neg(p \vee q) \equiv \neg p \wedge \neg q$
absorption	$p \vee (p \wedge q) \equiv p; p \wedge (p \vee q) \equiv p$

Type	Equivalent formulas
conditional	$p \rightarrow q \equiv \neg p \vee q$ $p \rightarrow q \equiv \neg q \rightarrow \neg p$ $(p \rightarrow q) \wedge (p \rightarrow r) \equiv p \rightarrow (q \wedge r)$ $(p \rightarrow q) \vee (p \rightarrow r) \equiv p \rightarrow (q \vee r)$ $(p \rightarrow r) \wedge (q \rightarrow r) \equiv (p \vee q) \rightarrow r$ $(p \rightarrow r) \vee (q \rightarrow r) \equiv (p \wedge q) \rightarrow r$
biconditional	$p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p)$ $p \leftrightarrow q \equiv \neg p \leftrightarrow \neg q$ $p \leftrightarrow q \equiv (p \wedge q) \vee (\neg p \wedge \neg q)$

4.2 Solution workflow

1. **Translate:** define atoms and identify each connective.
2. **Structure:** find the main connective and useful subformulas.
3. **Choose a method:** truth table, counterexample, or equivalence laws.
4. **Show the reason:** label the decisive row or each algebraic step.
5. **Check:** read the result back in the original setting.

Use a truth table for exhaustive verification, a counterexample for quick disproof, and algebra when the formula has useful structure.

4.3 Practice

1. Which are propositions: $2^{10} = 1024$; “Close the browser”; x is prime; every integer greater than 1 is a product of primes?
2. Build the truth table for $(p \wedge q) \vee \neg p$ and classify it.
3. Use a truth table to show $\neg(p \vee q) \equiv \neg p \wedge \neg q$.
4. Find the unique assignment that makes $p \rightarrow q$ false.
5. Translate “the service starts only if the database is available.”
6. Explain the difference between $A \leftrightarrow B$ and $A \equiv B$.
7. Simplify $(p \rightarrow q) \wedge p$.
8. On an island, every inhabitant is either a knight, who always tells the truth, or a knave, who always lies. Let

$$p = \text{“A is a knight”}, \quad q = \text{“B is a knight”}.$$

Let S_A denote the statement made by A, and let S_B denote the statement made by B. The island rule gives

$$p \leftrightarrow S_A, \quad q \leftrightarrow S_B.$$

Determine every possible type assignment in each exchange:

- (a) A says “B is a knight.” B says “A is a knight.”
- (b) A says “B is a knave.” B says “A is a knave.”
- (c) A says “B is a knave.” B says “We are opposite types.”

Answers

1. The first and fourth are propositions; the second is a command; the third has a free variable.
2. The formula is false only when $p = \text{T}, q = \text{F}$, so it is a contingency.
3. The two final columns are F, F, F, T in the standard row order.
4. $p = \text{T}, q = \text{F}$.
5. If s means “service starts” and d means “database available”, then $s \rightarrow d$.
6. $A \leftrightarrow B$ is a formula evaluated on one row; $A \equiv B$ asserts agreement on every row.
7. $(p \rightarrow q) \wedge p \equiv (\neg p \vee q) \wedge p \equiv p \wedge q$.
8. A knight’s statements are true, and a knave’s statements are false. Therefore, a speaker’s type and the truth value of that speaker’s statement agree.
 - (a) Here $S_A = q$ and $S_B = p$, so

$$(p \leftrightarrow q) \wedge (q \leftrightarrow p) \equiv p \leftrightarrow q.$$

The biconditional means that p and q agree, so the possible assignments are

$$(p \wedge q) \vee (\neg p \wedge \neg q).$$

Thus A and B have the same type: both knights or both knaves. The exchange does not determine which of these assignments is the actual one.

- (b) Here $S_A = \neg q$ and $S_B = \neg p$, so

$$(p \leftrightarrow \neg q) \wedge (q \leftrightarrow \neg p) \equiv (p \wedge \neg q) \vee (\neg p \wedge q) \equiv p \oplus q.$$

Thus A and B have opposite types. The exchange again does not determine which person is the knight.

- (c) Here $S_A = \neg q$ and $S_B = p \oplus q$, so

$$(p \leftrightarrow \neg q) \wedge (q \leftrightarrow (p \oplus q)).$$

If p is true, A’s statement forces q to be false. Then $p \oplus q$ is true, which contradicts $q \leftrightarrow (p \oplus q)$. Therefore p is false. A’s false statement forces q to be true. B’s statement $p \oplus q$ is then true, so this assignment is consistent. Thus

$$\boxed{\neg p \wedge q};$$

A is a knave and B is a knight.

5 Summary

The concepts form a dependency chain:

$$\begin{aligned} \text{propositions} &\longrightarrow \text{formulas} \longrightarrow \text{truth tables} \longrightarrow \text{equivalence} \\ &\longrightarrow \text{implication and biconditional} \longrightarrow \text{algebraic reasoning.} \end{aligned}$$

The central habit is simple: identify the smallest statements, build the formula, and ask what happens under every possible assignment.